

TP noté n° 1 — Durcissement d'un serveur Debian 12

DURCISSEMENT LINUX ET WINDOWS

5E ANNÉE

TRAVAIL INDIVIDUEL

Lancement en séance 4 · **rendu avant le début de la séance 5** · 50 % de la note du module

Le scénario

Vous reprenez l'administration d'un serveur Debian 12 qui héberge une application métier. Il n'a jamais été correctement administré : comptes partagés, droits laxistes, services inutiles, protections noyau désactivées, journalisation inexploitable.



Votre mission : ramener ce serveur à un niveau de sécurité défendable sans interrompre le service métier.

Cette dernière contrainte est le cœur du sujet. Un serveur éteint est parfaitement sécurisé et parfaitement inutile.



Cadre légal. Toutes les manipulations restent confinées à votre machine virtuelle, en réseau NAT ou hôte seul. Toute action sur un système tiers relève des articles 323-1 et suivants du code pénal. Note de zéro et signalement.

Ce que vous recevez

Fichier	Rôle
<code>degrade-debian.sh</code>	Installe l'état de départ. Tout le monde part du même point.
<code>check-debian.sh</code>	Le script qui vous corrigera. Il est public, lancez-le autant de fois que vous voulez.

- ✔ Douze des vingt points sortent de `check-debian.sh`. Vous connaissez donc votre note automatisée avant de rendre.

Étape 0 — Mise en place

🕒 AVANT LA SÉANCE 4 · ENVIRON 1 H

1 Installer un hyperviseur

VirtualBox, VMware, ou **UTM sur Mac Apple Silicon**.

2 Installer une Debian 12 minimale

Image netinst sur [debian.org](https://deb.debian.org/debian/netinst). 2 Go de RAM, 20 Go de disque, réseau NAT. Sur Apple Silicon, prendre l'image **ARM64** : tout le sujet fonctionne à l'identique.

3 Créer votre compte

Activer `sudo`, installer `openssh-server`.

4 Exécuter le script de dégradation

Copier `degrade-debian.sh` dans la VM et l'exécuter **en root**. Taper `OUI`.

5 Prendre un instantané nommé `baseline`

Obligatoire. C'est votre filet de sécurité et la preuve de votre point de départ.

6 Vérifier le service métier

`curl http://127.0.0.1:8080/health` doit renvoyer `OK`.

Vous allez vous verrouiller hors de votre machine au moins une fois pendant ce TP. C'est normal, c'est même formateur. L'instantané est ce qui transforme l'incident en anecdote.

Étape 1 — Audit initial

☆ 1,5 POINT

🕒 ENVIRON 1 H

Auditer l'état de départ avec **au moins deux outils**, dont Lynis. Conserver les rapports bruts : ils prouvent votre point de départ.

Produire ensuite une **liste priorisée** des faiblesses. Pour chacune : la constatation factuelle, la menace associée, et la criticité que vous lui attribuez **avec votre argument**.

⚠ **Le piège** : recopier la sévérité affichée par l'outil. Un outil ne connaît pas votre serveur. Une faiblesse critique selon Lynis peut être sans objet ici, et l'inverse est vrai aussi. C'est votre priorisation qui est notée, pas celle du logiciel.

Livrable

audit-avant.txt et la section « audit » du rapport.

Étape 2 — Comptes et authentification

 ENVIRON 1 H



Inventaire

Qui existe, qui peut se connecter, qui peut devenir root par n'importe quel chemin.



Nettoyage

Supprimer ou verrouiller les comptes superflus. Retirer le shell interactif des comptes qui n'ouvrent jamais de session.



Politique d'expiration

Mettre en place une politique d'expiration des mots de passe.



Robustesse

Ajouter un module de qualité de mot de passe.



Verrouillage après échecs répétés

Configurer le verrouillage automatique du compte.



Avertissement. Les points 4 et 5 touchent à PAM, qui s'applique à *tout* ce qui authentifie. Une erreur vous verrouille entièrement, root compris. Gardez une seconde session root ouverte dans un autre terminal pendant toute la manipulation, et ne la fermez qu'après avoir vérifié qu'une nouvelle connexion fonctionne.

Étape 3 — Élévation de privilèges

🕒 ENVIRON 30 MIN

Réécrire **entièrement** les règles `sudo` selon le moindre privilège, et activer la journalisation des commandes.

Avant d'autoriser une commande, posez-vous une seule question : **est-ce que ce binaire sait exécuter autre chose ?** Beaucoup d'outils du système de base savent lancer un shell. Autoriser une seule commande peut suffire à donner root.

Livrable

Votre fichier de règles, et dans le rapport, la justification de chaque autorisation accordée.

Étape 4 — Accès distant

🕒 ENVIRON 45 MIN

Durcir la configuration SSH, basculer sur l'authentification par clé, restreindre les comptes autorisés à se connecter.



Avertissement 1

Déposez votre clé publique et **testez une nouvelle connexion** avant de couper l'authentification par mot de passe. Dans l'autre ordre, vous restaurez l'instantané.



Avertissement 2

OpenSSH retient **la première valeur obtenue** pour chaque paramètre. Si un fichier de configuration antérieur définit déjà un paramètre, en ajouter un nouveau plus loin ne changera rien. Lisez l'ordre de chargement avant de vous acharner.



Vérification : `sshd -T` affiche la configuration réellement appliquée. C'est la seule source de vérité.

Étape 5 — Services, réseau et pare-feu

 ENVIRON 1 H

1. Réduire la surface d'écoute

Inventorier ce qui écoute, supprimer les services inutiles, désinstaller les protocoles obsolètes. À la fin, seuls les ports strictement nécessaires doivent être joignables.

2. Mettre en place un pare-feu

Configurer une politique par défaut restrictive en entrée, et des autorisations explicites.



Validez votre jeu de règles avant de l'appliquer. Un pare-feu qui bloque votre propre session SSH sur une machine distante est une leçon coûteuse.

Étape 6 — Système de fichiers

 ENVIRON 30 MIN



Permissions incorrectes

Corriger les permissions incorrectes sur les fichiers et répertoires sensibles.



Binaires SUID et SGID

Inventorier les binaires SUID et SGID, **justifier chacun** de ceux que vous conservez, supprimer les autres.



Fichiers accessibles en écriture

Traiter les fichiers accessibles en écriture à tous.



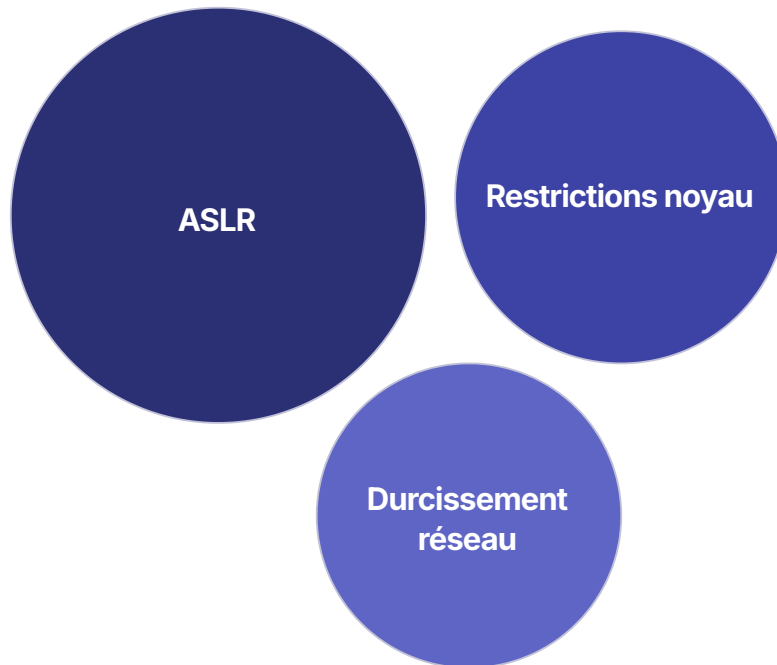
Options de montage

Examiner et durcir les options de montage des systèmes de fichiers.

Étape 7 — Noyau

🕒 ENVIRON 45 MIN

Réactiver les protections désactivées : randomisation des adresses, restrictions d'accès aux informations du noyau, durcissement de la pile réseau.



⚠️ **Ce qui est noté ici, ce n'est pas la liste, c'est la justification.** Pour chaque paramètre, une phrase : *qu'est-ce que ce réglage retire à l'attaquant ?* Une liste copiée depuis un site sans justification ne rapporte rien.

Attention également à l'ordre de chargement des fichiers de configuration : contrairement à SSH, ici c'est **la dernière valeur** qui l'emporte.

Étape 8 — Confinement des processus

🕒 ENVIRON 1 H

1

Remettre en route le MAC

Remettre en route le contrôle d'accès obligatoire (AppArmor).

2

Passer en mode enforce

Passer les profils existants en mode `enforce`.

3

Couvrir le service métier

Créer un profil de confinement dédié au service métier.

4

Renforcer systemd

Renforcer le confinement de l'unité `systemd` du service.



C'est ici que le service métier tombe le plus souvent. Méthode obligatoire : profil en `complain`, faire vivre l'application, lire les refus journalisés, compléter, et seulement là basculer en `enforce`. Testez le service après **chaque** bascule.

Étape 9 — Journalisation et intégrité

🕒 ENVIRON 1 H

1

Journal persistant

Rendre le journal persistant et le dimensionner correctement.

2

Audit noyau

Installer un système d'audit noyau avec **un jeu de règles motivé**. Chaque règle doit répondre à une question d'enquête précise que vous énoncez dans le rapport.

3

Intégrité des fichiers

Mettre en place un outil d'intégrité des fichiers et créer sa base de référence.



Question à traiter explicitement dans le rapport : où stockez-vous la base de référence ? Sur la machine surveillée, elle ne vaut rien.

Étape 10 — Mises à jour

 ENVIRON 20 MIN

Correctifs en attente

Appliquer les correctifs de sécurité en attente sur le système.

Mises à jour automatiques

Configurer les mises à jour automatiques de sécurité pour maintenir le système à jour sans intervention manuelle.

Étape 11 — Vérification

 ENVIRON 30 MIN

1 Redémarrer la machine

Tout doit repartir seul.

2 Vérifier le service métier

`curl http://127.0.0.1:8080/health` doit renvoyer `OK`.

3 Relancer `check-debian.sh`

Conserver la sortie dans `check-apres.txt`.

4 Relancer Lynis

Comparer avec le rapport initial pour mesurer la progression.

 **Si le service métier ne répond pas, votre note est plafonnée à 8/20**, quel que soit le reste du travail. Vous pouvez vérifier ce point autant de fois que vous voulez : il n'y a aucune raison d'être surpris.

Étape 12 — Rapport

☆ PARTIE DIFFÉRENCIANTE

🕒 ENVIRON 2 H

Huit pages maximum, annexes non comptées, dans cet ordre :

1

Synthèse de l'audit initial

La liste priorisée de l'étape 1.

2

Tableau des mesures appliquées

Pour chacune : la menace couverte, l'impact fonctionnel évalué, la vérification effectuée.

3

Mesures volontairement écartées

Au moins deux, avec l'argument : coût, régression, menace non pertinente ici. *Chaque année, la moitié des rendus laisse cette section vide. C'est 1,5 point perdu et un signal négatif sur la compréhension.*

4

Incidents rencontrés

Et comment vous les avez résolus. Vous verrouiller hors de votre machine n'est pas une faute, ne pas savoir expliquer pourquoi en est une.

5

Risque résiduel

Ce contre quoi ce serveur n'est toujours pas protégé.

Étape bonus — Démonstration de défense

✧ FACULTATIF · JUSQU'À +2 POINTS

🕒 NE PAS DÉPASSER 30 MIN

Au choix, sur votre machine virtuelle uniquement :

ASLR

Montrer, adresses à l'appui, l'effet de la randomisation d'adresses sur un programme C de dix lignes.

Profil de confinement

Écrire un profil de confinement pour un binaire et démontrer qu'il bloque un accès non prévu.

Filtrage syscall

Filtrer les appels système d'un programme simple et montrer le blocage.



Ce qui est attendu, c'est la démonstration de l'efficacité d'une défense, pas un exploit. Un exploit recopié ne rapporte aucun point. Trois lignes de terminal, une explication du mécanisme, et le bonus est acquis en entier.

Livrables

Archive NOM-Prenom-TP1.zip ou dépôt Git privé partagé :

rapport.pdf 8 pages maximum
hardening.sh votre script de durcissement, commenté et idempotent
audit-avant.txt rapport d'audit initial
audit-apres.txt rapport d'audit final
check-apres.txt sortie de check-debian.sh
configs/ vos fichiers de configuration modifiés

⊗ **Votre durcissement doit être scripté.** `hardening.sh` doit être rejouable deux fois de suite sur une baseline neuve sans rien casser. Un durcissement tapé à la main ne se reproduit pas et ne s'audite pas — et il coûte 2 points.

Barème

Critère	Points
Score de <code>check-debian.sh</code> , converti selon la table de la grille	12
Audit initial priorisé et argumenté	1,5
Justification des mesures	2,5
Mesures écartées	1,5
Qualité et idempotence du script	1,5
Rapport	1
Bonus démonstration	+2

⊗ **Plafond à 8/20 si le service métier ne répond plus.** Malus de 2 points par jour de retard entamé, note nulle au-delà de trois jours.

Travail individuel. L'entraide sur l'installation est encouragée, le durcissement et le rapport sont personnels. Deux rendus dont les scripts et les justifications se recouvrent significativement sont ramenés à zéro tous les deux. Les sources externes sont autorisées et recommandées, à condition d'être citées. Les outils d'IA générative sont autorisés et **à déclarer en annexe** ; ce qui est noté est votre compréhension, et une soutenance peut le vérifier.